

15 · Triggers

Why this matters in interviews. "What is a trigger? Row-level vs statement-level?", "what are `:OLD` and `:NEW`?", "in what order do triggers fire?", "what is the mutating-table error and how do you avoid it?", "write an audit trigger" — triggers connect PL/SQL to data changes, and are a favourite for scenario questions.

In this chapter: what triggers are and when to use them · syntax · `BEFORE` / `AFTER` / `INSTEAD OF` · row vs statement level · `:OLD` / `:NEW` · firing order · conditional predicates · audit, validation and view triggers · compound triggers and the **mutating-table** problem · managing triggers.

Sample table (used in the examples): `employees(employee_id, first_name, salary, department_id)`.

15.1 What is a trigger?

Triggers are stored programs that are automatically executed — "fired" — when some event occurs. They are written to run in response to any of these events:

Event class	Examples
A database manipulation (DML) statement	<code>DELETE</code> , <code>INSERT</code> , <code>UPDATE</code>
A database definition (DDL) statement	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code>
A database operation	<code>SERVERERROR</code> , <code>LOGON</code> , <code>LOGOFF</code> , <code>STARTUP</code> , <code>SHUTDOWN</code>

Triggers can be defined on the **table, view, schema or database** with which the event is associated.



What triggers are for

- Generating some **derived column values** automatically
- **Enforcing referential integrity** (beyond what constraints can express)
- **Event logging** and storing information on table access
- **Auditing** — who changed what, and when
- **Synchronous replication** of tables
- Imposing **security authorisations**
- **Preventing invalid transactions**
- **Enforcing business rules** and validation
- **Maintaining derived data** (auto-calculating totals) · **automatically updating related tables**

Use the simplest tool that works. A `NOT NULL` , `CHECK` , `UNIQUE` or foreign key constraint is faster, clearer and harder to bypass than a trigger. Use triggers for rules constraints **cannot** express (cross-table checks, auditing, derived values). Triggers are invisible to application developers — the classic source of "why did my update do that?".

15.2 Syntax

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF}
{INSERT [OR] | UPDATE [OR] | DELETE}
[OF col_name]
ON table_name
[REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW]
[WHEN (condition)]
DECLARE
    -- variable declarations
BEGIN
    -- trigger logic
EXCEPTION
    -- optional exception handling
END;
/
```

Clause	Meaning
CREATE [OR REPLACE] TRIGGER trigger_name	creates the trigger, or replaces an existing one with that name
BEFORE AFTER INSTEAD OF	when the trigger executes. INSTEAD OF is used for triggers on a view
INSERT [OR] UPDATE [OR] DELETE	the DML operation(s) that fire it (combine with OR)
OF col_name	fire only when that column is updated
ON table_name	the table (or view) the trigger is attached to
REFERENCING OLD AS o NEW AS n	lets you rename the old/new correlation names used for INSERT, UPDATE and DELETE (:o.salary , :n.salary)
FOR EACH ROW	makes it a row-level trigger — it runs once for each row affected . Without it, it is a statement-level (table-level) trigger that runs once per statement
WHEN (condition)	only fire for rows meeting the condition — valid only for row-level triggers ; column names here are written without the colon
DECLARE ... BEGIN ... EXCEPTION ... END;	an ordinary PL/SQL body

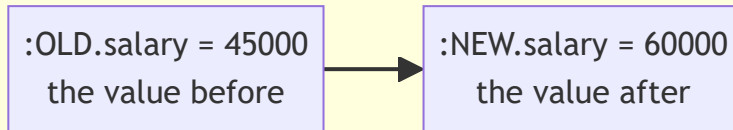
:OLD and :NEW

Inside a **row-level** trigger the two pseudo-records **:OLD** and **:NEW** hold the row's **before** and **after** images:

Event	:OLD.col	:NEW.col
INSERT	NULL (there was no row)	the values being inserted
UPDATE	the values before the update	the values after the update

Event	:OLD.col	:NEW.col
DELETE	the values of the row being deleted	NULL

UPDATE employees SET salary = 60000 WHERE employee_id = 103



You reference them **with a colon** inside the body (`:NEW.first_name`) and **without** in the `WHEN` clause. In a **BEFORE** row trigger you may **assign** to `:NEW` to change what is stored (`:NEW.email := LOWER(:NEW.email);`); `:OLD` is read-only, and in an **AFTER** trigger neither can be changed.

Error ORA-04082: NEW or OLD references not allowed in table level triggers. You used `:OLD` / `:NEW` in a **statement-level** trigger. Fix: add **FOR EACH ROW**.

15.3 Kinds of triggers

By timing

Type	Fires	Typical use
BEFORE trigger	before the DML operation	validate or modify data before it is saved
AFTER trigger	after the DML operation	logging, auditing , cascading changes to other tables
INSTEAD OF trigger	on a view , <i>instead of</i> the DML	define what DML on a view means

By event

INSERT (a new record is inserted) · **UPDATE** (a record is updated) · **DELETE** (a record is deleted). They can be **combined**: `AFTER INSERT OR UPDATE OR DELETE ON employees`.

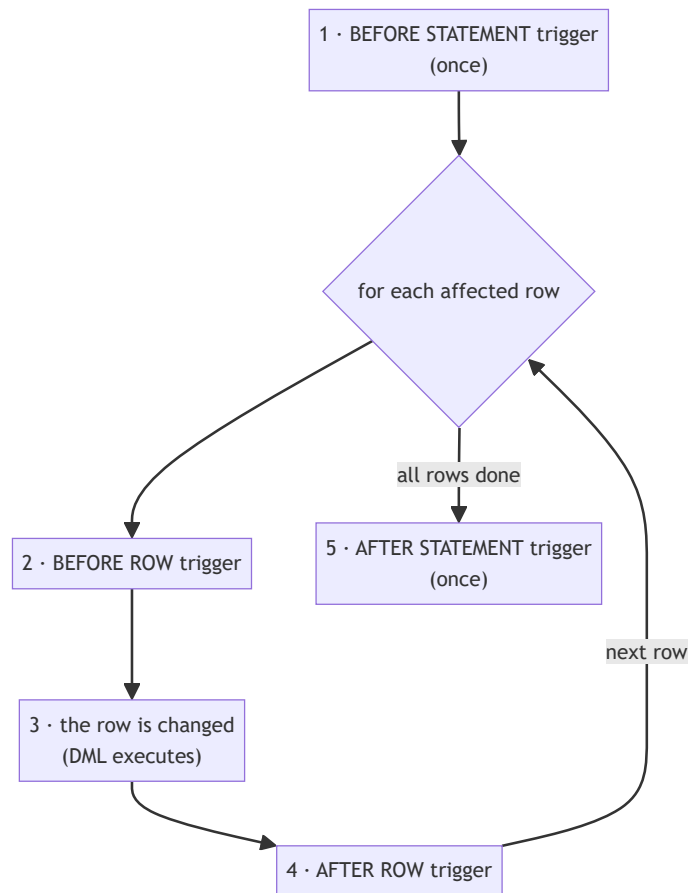
By level

Type	Fires	Keyword
Row-level	once for each affected row	<code>FOR EACH ROW</code>
Statement-level	once per statement , regardless of how many rows it affected — even if it affected zero	(<i>no</i> <code>FOR EACH ROW</code>)

Combining timing and level gives **four classic points** for each DML event: **before statement, before row, after row, after statement**.

15.4 Firing order

For a single DML statement affecting several rows:



For a statement that updates **two** rows, with one trigger of each of the four kinds, the output would be:

```
BEFORE statement
  BEFORE row (row 1)
  AFTER row (row 1)
  BEFORE row (row 2)
  AFTER row (row 2)
AFTER statement
```

(Some references list the five steps per *single row*: BEFORE statement → BEFORE row → DML → AFTER row → AFTER statement — the same sequence for a one-row statement.) If several triggers of the same kind exist on a table their relative order is **undefined** unless you specify it with `FOLLOWS` (Oracle 11g+).

15.5 Worked examples

1 · A simple `AFTER INSERT` row-level trigger

```
CREATE OR REPLACE TRIGGER trg_after_insert_emp
AFTER INSERT
ON employees
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('New employee added: ' || :NEW.first_name);
END;
/
```

- `AFTER INSERT` fires after a new row is inserted; `FOR EACH ROW` runs once per inserted row.
- `:NEW` refers to the **new values** being inserted; `:OLD` (not available in an `INSERT`) would refer to old values.

```
SET SERVEROUTPUT ON;
INSERT INTO employees (employee_id, first_name) VALUES (7, 'Kavya');
```

New employee added: Kavya

1 row inserted.

2 · A combined-event statement-level trigger

```
CREATE OR REPLACE TRIGGER trg_combined
AFTER INSERT OR UPDATE OR DELETE
ON employees
BEGIN
    DBMS_OUTPUT.PUT_LINE('Row modified in EMPLOYEES table');
END;
/
```

No `FOR EACH ROW`, so it prints **once per statement** — even a `DELETE` of 1,000 rows prints one line.

3 · Row-level versus statement-level side by side

```
-- Row-level trigger
CREATE OR REPLACE TRIGGER trg_row_level
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Row Trigger fired');
END;
/

-- Statement-level trigger
CREATE OR REPLACE TRIGGER trg_statement_level
AFTER INSERT OR UPDATE ON employees
BEGIN
    DBMS_OUTPUT.PUT_LINE('Statement Trigger fired');
END;
/
```

Run `UPDATE employees SET salary = salary + 1 WHERE department_id = 20;` where 3 rows match:

```
Row Trigger fired
Row Trigger fired
Row Trigger fired
Statement Trigger fired
```

The row trigger fires **three times** (once per row); the statement trigger **once**, after all rows.

4 · Conditional predicates — INSERTING, UPDATING, DELETING

Inside a **multi-event** trigger, use these Boolean functions to find out which operation fired it:

```
IF INSERTING THEN
    DBMS_OUTPUT.PUT_LINE('Insert occurred');
ELSIF UPDATING THEN
    DBMS_OUTPUT.PUT_LINE('Update occurred');
ELSIF DELETING THEN
    DBMS_OUTPUT.PUT_LINE('Delete occurred');
END IF;
```

`UPDATING('salary')` is TRUE only if that particular column is in the `SET` list.

5 · An audit trigger

Log every change to `employees` into an audit table:

```

CREATE TABLE emp_audit (
    emp_id      NUMBER,
    action      VARCHAR2(10),
    action_date DATE
);

CREATE OR REPLACE TRIGGER trg_emp_audit
AFTER INSERT OR UPDATE OR DELETE
ON employees
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO emp_audit VALUES (:NEW.employee_id, 'INSERT', SYSDATE);
    ELSIF UPDATING THEN
        INSERT INTO emp_audit VALUES (:OLD.employee_id, 'UPDATE', SYSDATE);
    ELSIF DELETING THEN
        INSERT INTO emp_audit VALUES (:OLD.employee_id, 'DELETE', SYSDATE);
    END IF;
END;
/

```

This trigger **logs every change** made to `employees` into `emp_audit`. Note the choice: on `INSERT` there is no `:OLD` row, on `DELETE` no `:NEW` row — so each branch uses the image that exists.

6 · Salary history — BEFORE UPDATE OF salary

Keep the old and new salary each time it changes (the lab's `store_history`):

```

CREATE OR REPLACE TRIGGER store_history
BEFORE UPDATE OF salary
ON emp
FOR EACH ROW
BEGIN
    INSERT INTO emp_sal_history (emp_id, old_salary, new_salary, change_date)
    VALUES (:OLD.emp_id, :OLD.salary, :NEW.salary, SYSDATE);
END;
/

-- ORA-04082: NEW or OLD references not allowed in table level triggers
-- for the above error try: FOR EACH ROW

UPDATE emp SET salary = 60000 WHERE emp_id = 103;

```

`OF salary` limits firing to updates that **include the salary column**; an update of only `job` will not fire it.

7 · Validation and automatic values — BEFORE triggers

A `BEFORE` row trigger can **reject** a change or **fix** the data before it is stored:

```

CREATE OR REPLACE TRIGGER trg_emp_rules
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    -- reject invalid data: a custom error, number range -20000 .. -20999
    IF :NEW.salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be negative');
    END IF;

    -- fix data before it is saved
    :NEW.first_name := INITCAP(:NEW.first_name);

    -- default a value for new rows only
    IF INSERTING AND :NEW.salary IS NULL THEN
        :NEW.salary := 30000;
    END IF;
END;
/

```

```

INSERT INTO employees (employee_id, first_name, salary) VALUES (8, 'MEERA', -5);
-- ORA-20001: Salary cannot be negative

```

`RAISE_APPLICATION_ERROR(-20xxx, message)` aborts the statement **and rolls back that statement's changes** — the standard way for a trigger to *prevent an invalid transaction*.

8 • INSTEAD OF — making a view writable

Views don't store data, so you can't normally `INSERT`, `UPDATE` or `DELETE` through a **join view**. **INSTEAD OF triggers** define what should happen when you do:

```

CREATE OR REPLACE VIEW emp_dept_vw AS
SELECT e.employee_id, e.first_name, d.department_name
FROM   employees e
JOIN   departments d ON e.department_id = d.department_id;

CREATE OR REPLACE TRIGGER trg_instead_of_insert
INSTEAD OF INSERT ON emp_dept_vw
FOR EACH ROW
BEGIN
    INSERT INTO employees (employee_id, first_name, department_id)
    VALUES (:NEW.employee_id,
            :NEW.first_name,
            (SELECT department_id FROM departments
             WHERE department_name = :NEW.department_name));
END;
/

```

```

INSERT INTO emp_dept_vw VALUES (9, 'Nikhil', 'Sales'); -- now legal: the trigger routes it

```

INSTEAD OF triggers are always **row-level** and exist only on views.

15.6 Compound triggers and the mutating-table problem

The problem — `ORA-04091: table ... is mutating`

A **row-level** trigger fires *while* the table is being changed, so the table is in an **inconsistent state**. If that trigger tries to **query or modify the same table** that fired it, Oracle raises `ORA-04091: table SCOTT.EMPLOYEES is mutating, trigger/function may not see it`.

```
-- X Fails on UPDATE: the row trigger reads the very table being updated
CREATE OR REPLACE TRIGGER trg_bad
AFTER UPDATE ON employees
FOR EACH ROW
DECLARE
    v_total NUMBER;
BEGIN
    SELECT SUM(salary) INTO v_total FROM employees;      -- ORA-04091
END;
/
```

Before Oracle 11g the workaround was ugly: **package-level variables plus a separate statement-level trigger**, spread over several objects.

The solution — the compound trigger (Oracle 11g+)

A **compound trigger** is a **single trigger object that lets you define several timing points** (statement-level *and* row-level) **inside one PL/SQL unit**. The key benefit: you can **declare variables shared by all timing sections**, so you can **accumulate data on a per-row basis and act on it once per statement** — simpler code, and it avoids the mutating-table error.

```

CREATE [OR REPLACE] TRIGGER trigger_name
FOR INSERT OR UPDATE OR DELETE ON table_name
COMPOUND TRIGGER

    -- declaration area (shared across all sections)
    <declarations>

    BEFORE STATEMENT IS
    BEGIN
        -- runs once before the DML statement
    END BEFORE STATEMENT;

    BEFORE EACH ROW IS
    BEGIN
        -- runs once per affected row, before the row change
    END BEFORE EACH ROW;

    AFTER EACH ROW IS
    BEGIN
        -- runs once per affected row, after the row change
    END AFTER EACH ROW;

    AFTER STATEMENT IS
    BEGIN
        -- runs once after the DML statement completes
    END AFTER STATEMENT;

END;
/

```

The recipe that avoids the error:

1. **Declare a PL/SQL collection** (or counters) in the **shared declaration area**.
2. In **BEFORE EACH ROW** or **AFTER EACH ROW**, **push the relevant values** into it — *don't touch the table*.
3. In **AFTER STATEMENT** — safe, because the DML has finished — **iterate the collection** and do the reads/writes on the same table, or bulk-process.

Practical example — log every changed employee once per statement, with a single bulk insert into the audit table instead of one insert per row:

```

CREATE OR REPLACE TRIGGER trg_emp_compound
FOR UPDATE ON employees
COMPOUND TRIGGER

    TYPE t_ids IS TABLE OF employees.employee_id%TYPE INDEX BY PLS_INTEGER;
    g_ids t_ids;                                -- shared by all sections

    AFTER EACH ROW IS
    BEGIN
        g_ids(g_ids.COUNT + 1) := :NEW.employee_id;    -- just remember the id
    END AFTER EACH ROW;

    AFTER STATEMENT IS
    BEGIN
        FORALL i IN 1 .. g_ids.COUNT                    -- one bulk insert, DML has finished
            INSERT INTO emp_audit VALUES (g_ids(i), 'UPDATE', SYSDATE);
    END AFTER STATEMENT;

END trg_emp_compound;
/

```

Rules: sections must appear in the order **before statement** → **before each row** → **after each row** → **after statement**, each **at most once**; the trigger has **no EXCEPTION** section of its own (put handlers inside each timing block); and `:OLD` / `:NEW` are available only in the **row** sections.

15.7 Managing triggers

```

ALTER TRIGGER trg_emp_audit DISABLE;           -- keep it, stop it firing (bulk loads)
ALTER TRIGGER trg_emp_audit ENABLE;
ALTER TABLE employees DISABLE ALL TRIGGERS;    -- every trigger on the table
ALTER TABLE employees ENABLE ALL TRIGGERS;
DROP TRIGGER trg_emp_audit;

SELECT trigger_name, trigger_type, triggering_event, table_name, status
FROM user_triggers;                            -- what exists
SELECT text FROM user_source WHERE name = 'TRG_EMP_AUDIT' AND type = 'TRIGGER' ORDER BY line;

```

Rules and pitfalls

Rule	Detail
No COMMIT / ROLLBACK inside a trigger	the trigger is part of the triggering statement's transaction — <code>ORA-04092: cannot COMMIT in a trigger</code> . (Only an <i>autonomous transaction</i> can commit independently — used for logging that must survive a rollback.)
Mutating table	don't read/modify the table that fired a row-level trigger (<code>ORA-04091</code>) — use a compound trigger

Rule	Detail
<code>:OLD / :NEW</code> need <code>FOR EACH ROW</code>	else <code>ORA-04082</code>
Triggers can cascade	a trigger's DML can fire other triggers — keep chains short and avoid loops
<code>TRUNCATE</code> does not fire <code>DELETE</code> triggers	it is DDL
Performance	row-level triggers run per row; a million-row update runs the body a million times. Keep the body small; batch with compound triggers
Testing	disable triggers during data loads <i>deliberately</i> and re-enable afterwards
Invalid triggers	a trigger with compile errors is INVALID and blocks DML on its table with <code>ORA-04098</code>

Interview corner

Q. What is a trigger? A stored PL/SQL program that runs automatically when a specified event occurs — a DML statement, a DDL statement or a database event such as logon.

Q. Types of triggers? By timing `BEFORE / AFTER / INSTEAD OF`; by event `INSERT / UPDATE / DELETE`; by level row vs statement; plus DDL/system triggers and compound triggers.

Q. Row-level vs statement-level? Row-level (`FOR EACH ROW`) fires once for every affected row and can use `:OLD / :NEW`; statement-level fires once per statement, even if no row was affected.

Q. What are `:OLD` and `:NEW`? Pseudo-records with a row's values before and after the change. `:OLD` is NULL on `INSERT`, `:NEW` is NULL on `DELETE`.

Q. `BEFORE` vs `AFTER` — when do you use which? `BEFORE` to validate or modify the incoming data (`:NEW` is writable); `AFTER` for logging, auditing and changes to other tables.

Q. In what order do triggers fire? Before statement → (for each row: before row → row change → after row) → after statement.

Q. What is an `INSTEAD OF` trigger? A row-level trigger on a view that replaces the DML, so views (especially join views) can be inserted into, updated and deleted from.

Q. What is the mutating-table error and how do you avoid it? `ORA-04091` — a row trigger read/modified the table that fired it. Avoid it by not touching that table in row triggers, or use a compound trigger: collect values per row, process them in `AFTER STATEMENT`.

Q. What is a compound trigger? A single trigger (11g+) with `BEFORE STATEMENT`, `BEFORE EACH ROW`, `AFTER EACH ROW` and `AFTER STATEMENT` sections that share declared variables.

Q. Can a trigger commit? No (`ORA-04092`), unless it uses an autonomous transaction.

Q. How do you stop an invalid change from a trigger? `RAISE_APPLICATION_ERROR(-20xxx, 'message')` — the statement fails and its changes roll back.

Q. Trigger vs constraint? Constraints declare simple rules the database enforces efficiently; triggers run code for rules constraints can't express. Prefer constraints when possible.

Q. How do you temporarily disable a trigger? `ALTER TRIGGER name DISABLE;` (or `ALTER TABLE ... DISABLE ALL TRIGGERS;`) and re-enable it afterwards.

Quick recap

- A **trigger** is a stored program **fired automatically** by DML, DDL or database events; defined on a table, view, schema or database.
- **Timing:** `BEFORE` (validate/modify `:NEW`) · `AFTER` (audit/cascade) · `INSTEAD OF` (views). **Level:** `FOR EACH ROW` (row, uses `:OLD` / `:NEW`) vs none (statement, once).
- `:OLD` = before, `:NEW` = after; `:OLD` NULL on insert, `:NEW` NULL on delete; `WHEN (...)` (no colon) is row-level only; `OF col` limits an `UPDATE` trigger.
- **Order:** before statement → before row → after row (per row) → after statement. `INSERTING` / `UPDATING` / `DELETING` tell a multi-event trigger what happened.
- **Mutating table** `ORA-04091` → compound trigger: collect in row sections, act in `AFTER STATEMENT`. `ORA-04082` → add `FOR EACH ROW`. No `COMMIT` in triggers (`ORA-04092`).
- Prefer constraints when they suffice; keep triggers short; `RAISE_APPLICATION_ERROR` rejects invalid changes.